A Fixed Iterated Greedy Sequence Cuts Weighted Tardiness on an Eight-Stage Flow Line by More than 2,500 Weighted Minutes per Period at Every Level of Processing-Time Variation Tested

OM 522 Team Project, Option 6

Muting (Don) Ma

University of Alabama, Fall 2026

November 2, 2026

Abstract. On a flow line where jobs cannot pass one another, a production planner must commit to one job order for every stage, and the earliest due date rule ignores job weights, release times, and the processing times at later stages. We applied the iterated greedy algorithm that Ruiz and Stützle proposed for flow shops with sequence-dependent setup times to an eight-stage line without setups but with release times and weighted due dates, and compared its fixed sequence with the earliest due date sequence on exact data and on 180 paired scenarios of random processing times. On development instances with 15, 45, and 120 jobs, iterated greedy lowered total weighted tardiness by 98.5, 97.6, and 81.0 percent, and a mixed-integer program solved with HiGHS proved its values optimal on the 6-job and 15-job instances. When each processing time of the 45-job instance varied with a coefficient of variation of 0.2, the iterated greedy plan had the lower weighted tardiness in all 30 scenarios, with a mean of 420.6 weighted minutes against 3,541.1, an 88.1 percent reduction. As the coefficient of variation rose from 0.1 to 0.8, the relative reduction fell from 94.6 to 27.2 percent while the mean absolute reduction stayed between 2,572.0 and 3,683.7 weighted minutes per period, and at 0.8 the 10th percentile of the per-period reduction fell to 851.5 and the earliest due date plan won 1 of 30 scenarios. A planner can therefore adopt the searched sequence for its absolute gain and, on the evidence of the one factor we varied, should expect that gain to arrive less dependably in a single period as processing times grow less predictable.

Contents

1 Background **2**

2 Data **3**

References **4**

1. Background

On a flow line such as printed circuit board assembly or furniture finishing, every job visits the same stages in the same order, and when jobs cannot pass one another, the order chosen at the first stage holds at every stage [1]. On the line studied here operations take 2 to 20 minutes, so a job that needs 2 minutes at the first stage and 20 at a later one would delay every job behind it at that later stage, and the waiting would build up downstream. The planner chooses one sequence for a period’s jobs, which are released at different times and carry due dates of different importance, expressed as weights. Vallada, Ruiz, and Minella report that dispatching rules are used very often in practice and that earliest due date is among the most common rules for total tardiness [2], and the course brief for this line makes it the rule that a researched method must beat [1]. Earliest due date orders the jobs by due date alone, so it cannot see a job’s weight, its release time, or its processing times at the stages after the first.

Minimizing total tardiness on a permutation flow shop is NP-hard in the ordinary sense with one machine and in the strong sense with two or more [2], and on this line an exact solver needed 85.8 seconds to certify the optimum for only 15 jobs, so a planner sequencing 45 to 120 jobs per period needs a heuristic. Ruiz and Stützle’s iterated greedy algorithm repeatedly removes a few jobs from a sequence and reinserts each at its best position [3], and on flow shops with sequence-dependent setup times and the weighted tardiness objective it had the lowest average deviation from the best known solutions among the methods they tested [4]. In a review of 40 methods for total tardiness, the heuristics built on inserting and interchanging jobs performed best among the heuristics, ahead of every dispatching rule [2]. These evaluations used instances with fixed processing times and no release times, and none of them executed a searched sequence on processing times that differ from the plan. A sequence tuned to the exact times in a data file could lose its advantage once the 2- to 20-minute operations run long or short, and a planner who must fix the order in advance cannot tell from these studies whether it does.

We implemented the earliest due date rule exactly as the course brief states it [1] and applied iterated greedy with local search [4] to an eight-stage line with release times and no setup times. A mixed-integer program solved with HiGHS [5] certified the optimal values of the two smallest instances, and a checker that shares no code with either method scored every schedule. We then held each method’s sequence fixed and executed it on 30 scenarios in which every processing time was multiplied by an independent lognormal factor with mean 1 and a coefficient of variation of 0.2, and on 30 further scenarios at each of five levels of variation from 0.1 to 0.8.

On exact data, iterated greedy left 1.5, 2.4, and 19.0 percent of the rule’s weighted tardiness on the instances with 15, 45, and 120 jobs, and it matched the proven optimum on the 6-job and 15-job instances. Held fixed on the 45-job instance at a coefficient of variation of 0.2, its plan left 11.5 jobs late per period on average, against 36.1 for the rule’s plan, and across all six sets of scenarios it had the lower weighted tardiness in 179 of 180. The mean reduction stayed between 2,572.0 and 3,683.7 weighted minutes at every level of variation, while the 10th percentile of the reduction in a single period fell from at least 1,781.1 weighted minutes at coefficients up to 0.5 to 851.5 at 0.8.

A planner can adopt the searched sequence for an absolute gain that held at every level of variation we tested, at the cost of a sequence that cannot be traced by hand and seconds of computation. Processing-time variability is the one condition we varied, and because it eroded the gain in the worst periods first, it is the first quantity to measure on a line before relying on the plan.

2. Data

The data are synthetic instances that the course instructor generated for option 6 of the OM 522 team project, with all times in minutes [1]. Each instance consists of a file of jobs, giving each job's release time, due date, and weight, a file of stages, giving their order on the line, and a file with one positive processing time for every job and stage. Table 1 summarizes the five instances: a 6-job case on 4 stages meant for tracing by hand, development instances with 15, 45, and 120 jobs on 8 stages, and a 10-job edge case on 4 stages.

Table 1: The five supplied instances. Processing times are per job and stage. The due-date window is the time from a job's release to its due date, and its median is taken over the instance's jobs. All times are in minutes.

Instance	Jobs	Stages	Processing times	Releases	Median due-date window	Weights
hand_checkable	6	4	3 to 19	8 to 78	30	1, 2, 4
development_small	15	8	2 to 20	24 to 210	158	1, 2, 4, 5
development_medium	45	8	2 to 20	10 to 607	154	1 to 5
development_large	120	8	2 to 20	3 to 1,690	154	1 to 5
edge_case	10	4	0 to 20	5 to 126	87.5	1, 2, 3, 5

The three development instances share one generator, so their processing times range from 2 to 20 minutes and average between 85.1 and 88.8 minutes of total processing per job over the 8 stages. Their releases spread over 186, 597, and 1,687 minutes, which gives arrival rates between 0.071 and 0.081 jobs per minute, so the larger instances cover longer planning periods at a similar density. The median time from release to due date is 154 to 158 minutes, and subtracting each job's own total processing time leaves a median of 64.5 to 74 minutes that a job can spend waiting before it is late. Weights are integer priority scores from 1 to 5, and a weighted minute is one minute of lateness multiplied by the job's priority [1].

Before solving, our tool applies the four checks the project guide requires: every required file and column is present, every value has its declared type, every job and stage referenced by a processing time exists, and every processing time is positive [6]. It reports each failure with its file, column, and line, and it never replaces a missing or invalid value. The edge case contains a processing time of zero, so the tool rejects it without writing a decision, as the guide specifies for this option [6].

References

[1] OM 522 instructor. OM 522 team project, option 6: Sequencing jobs through a common production line. Option brief and data package (schema version 4), University of Alabama, 2026. Course document, Fall 2026.

[2] Eva Vallada, Rubén Ruiz, and Gerardo Minella. Minimising total tardiness in the m-machine flowshop problem: A review and evaluation of heuristics and metaheuristics. *Computers & Operations Research*, 35(4):1350-1373, 2008. doi:10.1016/j.cor.2006.08.016.

[3] Rubén Ruiz and Thomas Stützle. A simple and effective iterated greedy algorithm for the permutation flowshop scheduling problem. *European Journal of Operational Research*, 177(3):2033-2049, 2007. doi:10.1016/j.ejor.2005.12.009.

[4] Rubén Ruiz and Thomas Stützle. An Iterated Greedy heuristic for the sequence dependent setup times flowshop problem with makespan and weighted tardiness objectives. *European Journal of Operational Research*, 187(3):1143-1159, 2008. doi:10.1016/j.ejor.2006.07.029.

[5] HiGHS development team. HiGHS: High performance software for linear optimization, version 1.15.1, 2026. URL: https://highs.dev.

[6] OM 522 instructor. OM 522 team project guide. Revised October 14, 2026, University of Alabama, 2026. Course document, Fall 2026.